SEMINARIO DE ARQUITECTURAS DE SOFTWARE

Un Análisis Comparativo de Estilos Basados en Capas y Dominio

Nombre del Equipo: LA ESCALONETA

Link del Proyecto:
https://github.com/CuellarMarlon/timeblock/tree/develop

Miembro	Responsabilidad / Rol
Raciel Pupo Santos	Preguntas 1 y 2
Ramón Hernández Camacho	Pregunta 3 e Informe
Leandro Márquez Blanco	Pregunta 4
Charly Blanco Sánchez	Pregunta 5
Marlon R. Cuellar García	Pregunta 6

Seminario de Arquitectura de Software: Preguntas del 1 al 6

Pregunta 1: Análisis de Trade-offs: Arquitectura N-Capas vs. Clean / Onion / Hexagonal

a) ¿Qué error fundamental de acoplamiento se está cometiendo?

En la Arquitectura N-Capas tradicional, el sistema se organiza en capas horizontales (Presentación, Negocio y Acceso a Datos) y cada capa depende de la que tiene debajo (Richards, 2015). En el sistema de inventario, esto significa que la interfaz de usuario conoce a la capa de negocio, y la capa de negocio conoce directamente a las clases concretas que comunican con SQL Server. Las dependencias del código fuente siguen el mismo camino que el flujo de control: de arriba hacia abajo, hasta terminar en la base de datos.

El error fundamental no es haber usado capas, sino la dirección de las dependencias. La lógica de negocio, que es la parte de más alto nivel y más valiosa del sistema (reglas como «no se puede despachar más stock del disponible» o «un producto por debajo del mínimo genera un pedido de reposición»), depende de la parte de más bajo nivel y más volátil: el motor de base de datos, el esquema de tablas y la librería de acceso a datos. Es exactamente lo contrario de lo que pide el Principio de Inversión de Dependencias: los módulos de alto nivel no deben depender de los de bajo nivel, sino que ambos deben depender de abstracciones (Martin, 2017). Martin (2017) lo resume diciendo que la base de datos es un detalle: un mecanismo para guardar datos que no debería condicionar las reglas de negocio.

Este error explica uno por uno los síntomas del enunciado:

- **Los cambios en la base de datos afectan al negocio y a la interfaz:** En teoría, el concepto de capas de aislamiento garantiza que un cambio en una capa no afecte a las demás, pero solo mientras el contrato entre ellas no cambie (Richards, 2015). Aquí el contrato lo define la capa inferior y es un reflejo del esquema: la capa de datos devuelve filas, DataSet o entidades del ORM mapeadas una a una con las tablas, y esas estructuras suben hasta la interfaz. Al renombrar una columna o dividir una tabla cambia el contrato y el cambio se propaga en cascada. Palermo (2008) señala precisamente este problema en la arquitectura en capas tradicional: cada capa queda acoplada a las capas inferiores y a cuestiones de infraestructura, y el acoplamiento de la interfaz y del negocio con el acceso a datos es el caso más común.
- **Las pruebas unitarias son lentas y frágiles:** Como la clase de negocio crea o recibe la clase concreta de acceso a datos, para probar una regla de inventario hay que tener SQL Server en marcha, con el esquema correcto y datos de prueba cargados. En realidad son pruebas de integración: son lentas porque hacen E/S, y frágiles porque fallan por motivos ajenos a la regla que prueban (datos residuales, cambios de esquema, problemas de conexión). Martin (2017) describe este fenómeno como el problema de las pruebas frágiles: las pruebas muy acopladas al sistema tienen que cambiar cada vez que cambia el sistema, y terminan siendo un freno en lugar de una ayuda.
- **El sistema se ha vuelto un monolito difícil de mantener:** La N-Capas hace una partición técnica: agrupa el código por su función técnica y no por el dominio, de modo que un concepto como

«movimiento de inventario» queda repartido entre todas las capas (Richards & Ford, 2020). Cualquier cambio en el dominio atraviesa todas las capas a la vez.

El siguiente fragmento muestra la forma típica que toma este acoplamiento:

```
// Capa de negocio (N-Capas tradicional)
public class InventoryService
{
    // Dependencia directa de una clase concreta que usa SQL Server.
    private readonly InventoryDal _dal = new InventoryDal();

    public void RegisterExit(int productId, int quantity)
    {
        DataRow row = _dal.GetProductRow(productId);   // estructura de la BD
        int stock = (int)row["StockActual"];           // nombre de columna
        if (quantity > stock)
            throw new InvalidOperationException("Stock insuficiente");

        _dal.UpdateStock(productId, stock - quantity);
    }
}
```

La regla de negocio (no despachar más de lo que hay) está mezclada con un DataRow, con el nombre de una columna y con una clase que solo funciona con SQL Server. Cambiar cualquiera de esos detalles obliga a tocar la lógica de negocio.

b) Cómo lo resuelven la Regla de Dependencia y los Puertos y Adaptadores

Regla de Dependencia (Martin): La Arquitectura Limpia organiza el sistema en círculos concéntricos: Entidades, Casos de Uso, Adaptadores de Interfaz, y Frameworks y Drivers. La regla que la hace funcionar es que las dependencias del código fuente solo pueden apuntar hacia adentro, hacia las políticas de más alto nivel; nada de un círculo interior puede conocer el nombre de algo declarado en un círculo exterior (Martin, 2017). Aplicado al inventario:

- **Entidades:** Product, Warehouse, StockMovement, con reglas como que el stock nunca puede quedar negativo.
- **Casos de uso:** RegisterExit, RegisterEntry, ListLowStockProducts.
- **Adaptadores de interfaz:** controladores, presentadores y las implementaciones concretas de los repositorios.
- **Frameworks y drivers:** ASP.NET, Entity Framework, el driver de SQL Server o de MongoDB.

El problema es que el caso de uso necesita guardar datos, y guardar datos ocurre en el círculo exterior. Martin (2017) resuelve esta contradicción con la inversión de dependencias: el flujo de control va hacia afuera, pero la dependencia del código fuente se invierte mediante polimorfismo. El caso de uso declara una interfaz con lo que necesita (IProductRepository) y la clase del círculo exterior la implementa. Además, los datos que cruzan la frontera deben ser estructuras simples o los propios objetos del dominio; no se deben pasar hacia adentro las «filas» que devuelve el framework de base de datos, porque eso obligaría al círculo interior a conocer al exterior (Martin, 2017).

Puertos y Adaptadores (Cockburn): La Arquitectura Hexagonal llega a la misma solución con otro vocabulario. La aplicación se comunica con el exterior únicamente a través de puertos, y por cada tecnología externa hay un adaptador que traduce entre la API del puerto y esa tecnología (Cockburn, 2005). En el inventario habría dos tipos de puertos:

- **Puertos de entrada (lado que dirige):** la API de la aplicación, por ejemplo «registrar salida de stock». La usan como adaptadores un controlador REST, una interfaz de escritorio, un proceso por lotes o una batería de pruebas automatizadas.
- **Puertos de salida (lado dirigido):** «guardar y recuperar productos». Sus adaptadores pueden ser uno para SQL Server, otro para MongoDB y otro en memoria para las pruebas.

Ambos enfoques solucionan el problema de forma estructural, no por disciplina del equipo. El proyecto del núcleo no tiene ninguna referencia a la librería de SQL Server, de modo que el compilador impide que la lógica de negocio use un DataRow o una cadena de conexión. Es el proyecto de infraestructura el que referencia al núcleo, y no al revés. Así la base de datos pasa a ser un «plugin» del negocio (Martin, 2017).

```
// ===== Núcleo (entidades y casos de uso) =====
public class Product
{
    public string Sku { get; }
    public int Stock { get; private set; }

    public Product(string sku, int stock) { Sku = sku; Stock = stock; }

    public void RegisterExit(int quantity)
    {
        if (quantity <= 0)
            throw new ArgumentException("La cantidad debe ser positiva");
        if (quantity > Stock)
            throw new InsufficientStockException(Sku, Stock, quantity);
        Stock -= quantity;
    }
}

// Puerto de salida: lo define el núcleo, en términos del dominio.
public interface IProductRepository
{
    Product? GetBySku(string sku);
    void Save(Product product);
}

// Caso de uso: solo conoce la abstracción.
public class RegisterExitUseCase
{
    private readonly IProductRepository _products;
    public RegisterExitUseCase(IProductRepository products) => _products = products;

    public void Execute(string sku, int quantity)
    {
        var product = _products.GetBySku(sku)
            ?? throw new ProductNotFoundException(sku);
```

```
        product.RegisterExit(quantity);
        _products.Save(product);
    }
}

// ===== Infraestructura (adaptador) =====
public class SqlServerProductRepository : IProductRepository
{
    // Única clase que conoce SQL Server, el esquema y las columnas.
    // ...
}
```

c) Impacto en la testabilidad y en el cambio de SQL Server a MongoDB

Testabilidad: El objetivo que Cockburn (2005) declara para su arquitectura es permitir que la aplicación sea dirigida por igual por usuarios, programas, pruebas automatizadas o scripts por lotes, y que se desarrolle y pruebe aislada de los dispositivos y bases de datos con los que funcionará en producción. Con el nuevo diseño eso se cumple en dos niveles:

- **1.** Las reglas de la entidad Product se prueban sin ningún doble de prueba, porque la clase no depende de nada externo.
- **2.** El caso de uso se prueba inyectando un adaptador en memoria en lugar del real.

```
public class InMemoryProductRepository : IProductRepository
{
    private readonly Dictionary<string, Product> _data = new();
    public Product? GetBySku(string sku) => _data.GetValueOrDefault(sku);
    public void Save(Product product) => _data[product.Sku] = product;
}

[Fact]
public void No_permite_despachar_mas_stock_del_disponible()
{
    var repo = new InMemoryProductRepository();
    repo.Save(new Product("SKU-001", stock: 5));
    var useCase = new RegisterExitUseCase(repo);

    Assert.Throws<InsufficientStockException>(
        () => useCase.Execute("SKU-001", quantity: 8));
}
```

Estas pruebas se ejecutan en milisegundos, no necesitan red ni servidor de base de datos y siempre dan el mismo resultado. Tampoco son frágiles ante cambios de esquema: si se renombra una columna, solo cambia el adaptador de SQL Server, y las pruebas del negocio siguen igual. Es lo que Martin (2017) presenta como una de las propiedades de la Arquitectura Limpia: las reglas de negocio pueden probarse sin la interfaz de usuario, la base de datos, el servidor web ni ningún otro elemento externo. Las pruebas de integración siguen siendo necesarias, pero se limitan a los adaptadores. Una buena práctica es escribir un único conjunto de pruebas del contrato IProductRepository y ejecutarlo contra cada adaptador (en memoria, SQL Server y MongoDB) para comprobar que todos se comportan igual.

Cambio de SQL Server a MongoDB: Con el diseño anterior, el cambio obligaría a reescribir la capa de datos, la de negocio (que usa DataRow y nombres de columnas) y probablemente la interfaz. Con puertos y adaptadores el cambio se reduce a tres pasos:

- **1.** Escribir un nuevo adaptador, MongoProductRepository, que implemente la interfaz IProductRepository usando el driver de MongoDB.
- **2.** Registrarlo en la raíz de composición en lugar del adaptador de SQL Server.
- **3.** Migrar los datos, que es una tarea operativa y no afecta al código del núcleo.

```
public class MongoProductRepository : IProductRepository
{
    private readonly IMongoCollection<ProductDocument> _collection;

    public MongoProductRepository(IMongoDatabase db)
        => _collection = db.GetCollection<ProductDocument>("products");

    public Product? GetBySku(string sku)
    {
        var doc = _collection.Find(d => d.Sku == sku).FirstOrDefault();
        return doc is null ? null : new Product(doc.Sku, doc.Stock);
    }

    public void Save(Product product) =>
        _collection.ReplaceOne(d => d.Sku == product.Sku,
            new ProductDocument { Sku = product.Sku, Stock = product.Stock },
            new ReplaceOptions { IsUpsert = true });
}

// Raíz de composición: el único lugar que cambia.
// services.AddScoped<IProductRepository, SqlServerProductRepository>();
services.AddScoped<IProductRepository, MongoProductRepository>();
```

Product, RegisterExitUseCase, los controladores y las pruebas unitarias no se modifican ni se vuelven a probar por este motivo. La clase ProductDocument pertenece al adaptador; el núcleo no sabe que existe.

Condiciones para que funcione: Esta independencia no es automática. Depende de que el puerto se haya diseñado bien:

- **Lenguaje del dominio:** El puerto debe hablar el lenguaje del dominio, no el de la base de datos. Si IProductRepository expusiera IQueryable, SQL o filas, la abstracción estaría filtrando el detalle y el cambio a MongoDB volvería a romper el núcleo.
- **Identidad:** Si el núcleo dependiera de un int autoincremental generado por SQL Server, el cambio a los ObjectId de MongoDB lo afectaría. Por eso en el ejemplo la identidad es el SKU, que pertenece al dominio.
- **Transacciones y consistencia:** Si un caso de uso modifica varios objetos dentro de una misma transacción de SQL Server, en MongoDB habrá que usar transacciones de varios documentos o rediseñar los límites de consistencia. Cuando el modelo se organiza en agregados, cada agregado se corresponde de forma natural con un documento, y el cambio es más sencillo.

- **Consultas e informes:** Las consultas con muchas uniones (joins) pueden necesitar modelos de lectura distintos en MongoDB. Eso afecta a los adaptadores, no a las reglas de negocio.

Balance de compromisos (trade-offs):

- **Simplicidad inicial:** N-Capas tradicional es alta (conocida y rápida); Clean/Hexagonal/Onion es menor (requiere interfaces, mapeos y raíz de composición).
- **Cantidad de código:** N-Capas tradicional requiere menor código; Clean/Hexagonal/Onion mayor (puertos, adaptadores, DTOs).
- **Testabilidad del negocio:** N-Capas es baja si depende de clases concretas; Clean/Hexagonal/Onion es alta (pruebas rápidas con adaptadores en memoria).
- **Cambio de base de datos:** N-Capas afecta a negocio e interfaz; Clean/Hexagonal/Onion se limita a un adaptador.
- **Evolución a largo plazo:** N-Capas se degrada a medida que crece el sistema; Clean/Hexagonal/Onion se mantiene porque el núcleo está protegido.

La decisión inicial del equipo no fue absurda. Richards y Ford (2020) valoran la arquitectura en capas como la más simple y la de menor coste, y Richards (2015) la recomienda como punto de partida cuando todavía no se sabe qué estilo conviene. Las fronteras completas tienen un coste, y Martin (2017) reconoce que el arquitecto debe decidir dónde y cuándo pagarlo. El problema es que el inventario creció y el equipo nunca invirtió la dependencia más importante: la del negocio hacia la base de datos.

La migración no tiene por qué hacerse de golpe. El primer paso puede darse dentro de la propia N-Capas: mover la definición de las interfaces de repositorio a la capa de negocio y dejar sus implementaciones en la capa de datos. Es lo que propone la Guía de Arquitectura N-Capas orientada al Dominio, donde los contratos de los repositorios se definen en la capa de dominio y se implementan en la capa de infraestructura de persistencia, y donde se usa inyección de dependencias para conectarlos (De la Torre et al., 2010). Con ese único cambio la dependencia ya apunta hacia el negocio, las pruebas pueden usar adaptadores en memoria y el sistema queda preparado para evolucionar hacia una arquitectura Limpia o Hexagonal completa.

Pregunta 2: Comparación de Estilos: la Simetría del Hexágono vs. la Jerarquía de las Capas

a) ¿Qué problema conceptual resuelve la simetría del hexágono?

Un diagrama de arquitectura no es solo una ilustración: transmite cómo se debe pensar el sistema. La representación de capas apiladas transmite un orden vertical, y ese orden tiene consecuencias que el hexágono intenta eliminar.

- **1. La pila de capas trata como cosas distintas lo que es el mismo problema:** Cockburn (2005) parte de una observación: la lógica de negocio suele filtrarse hacia la interfaz de usuario y, al mismo tiempo, quedar atada a la base de datos. En el dibujo en capas estos dos problemas aparecen en extremos opuestos (la interfaz arriba y la base de datos abajo) y parecen problemas diferentes que se resuelven con reglas diferentes. Cockburn sostiene que en realidad son el mismo problema: la

aplicación está enredada con entidades externas. La interfaz de usuario y la base de datos son, desde el punto de vista de la aplicación, dos actores externos más.

- **2. El dibujo vertical sugiere una dependencia hacia abajo:** En una pila, lo natural es leer que lo de arriba se apoya en lo de abajo. La base de datos aparece como la «base» o el cimiento del sistema, y eso lleva a diseñar a partir de las tablas: primero el esquema y después el negocio. Es el error que se analizó en la pregunta 1. La propia forma del diagrama enseña la dirección de dependencias equivocada.
- **3. Una pila solo tiene un arriba y un abajo:** En la N-Capas hay sitio para un solo tipo de cliente (arriba) y un solo tipo de recurso (abajo). No queda claro dónde dibujar las pruebas automatizadas, un proceso por lotes, otra aplicación que consume la API, una cola de mensajes o un servicio de correo. Terminan como cajas laterales o como «excepciones» del diagrama. Cockburn (2005) cuenta que eligió el hexágono justamente para tener espacio donde dibujar todos los puertos y adaptadores que hagan falta, sin estar limitado por un dibujo en capas unidimensional.
- **4. La simetría hace visible la sustituibilidad:** En el hexágono solo hay dos regiones: el interior (la aplicación) y el exterior (todo lo demás). Todos los actores externos son equivalentes: un usuario humano, una prueba automatizada, otro sistema, la base de datos o el servidor de correo se comunican con la aplicación de la misma manera, a través de un puerto y un adaptador. Así se ve de forma intuitiva que reemplazar la interfaz gráfica por una batería de pruebas es la misma operación que reemplazar SQL Server por una base de datos en memoria. Cockburn (2005) lo plantea como objetivo: crear la aplicación para que funcione sin interfaz y sin base de datos, de modo que se puedan ejecutar pruebas de regresión automatizadas, seguir trabajando cuando la base de datos no está disponible y conectar aplicaciones entre sí sin intervención del usuario.

¿Y la Arquitectura Onion? Palermo (2008) ya corrige el problema principal de la N-Capas: sus capas son concéntricas, todo el acoplamiento apunta hacia el centro y en el anillo exterior coloca juntas la interfaz de usuario, la infraestructura y las pruebas. Palermo reconoce además que comparte con la Arquitectura Hexagonal la misma premisa: sacar la infraestructura hacia afuera y escribir código adaptador para que no quede fuertemente acoplada al núcleo. En ese sentido, el anillo exterior de Onion es casi tan simétrico como el hexágono.

Sin embargo, la metáfora de capas sigue poniendo el énfasis en otro lugar:

- • Las capas transmiten niveles y profundidad (Modelo de Dominio, Servicios de Dominio, Servicios de Aplicación), mientras que el hexágono transmite una frontera. Onion prescribe cómo organizar el interior; Cockburn deliberadamente no dice nada de él.
- • En Onion el concepto de puerto no aparece de forma explícita: las interfaces existen, pero el diagrama no muestra que cada una corresponde a una conversación con un propósito distinto ni que puede tener varios adaptadores.
- • En la práctica, la cebolla suele traducirse a una lista de proyectos (Core, Infrastructure, Web) que vuelve a leerse de arriba hacia abajo, y es fácil que el equipo recaiga en la intuición de la pila.

Por eso ambos estilos son complementarios más que rivales: el hexágono explica la relación de la aplicación con su entorno, y la cebolla explica cómo ordenar el interior.

b) Puerto y Adaptador en la práctica de Onion

La regla fundamental de Onion es que todo el código puede depender de las capas más centrales, pero nunca de las más externas. Las interfaces se definen en el núcleo y sus implementaciones se sitúan en el borde de la aplicación; como el núcleo necesita esas implementaciones, se inyectan en tiempo de ejecución (Palermo, 2008). Traducido al vocabulario de Cockburn:

- **Puerto =** una interfaz declarada en las capas internas (Modelo de Dominio, Servicios de Dominio o Servicios de Aplicación).
- **Adaptador =** una clase del anillo exterior (Infraestructura o Interfaz de Usuario) que implementa esa interfaz o que la invoca.

En la práctica esto se concreta en cuatro decisiones de diseño:

- **1. Cada tipo de puerto tiene su capa:** Los repositorios de agregados (IProductRepository) se declaran junto al modelo de dominio. Las necesidades técnicas de la aplicación (INotificationSender, IClock) se declaran en servicios de aplicación. Los servicios de aplicación (IInventoryService) son la API que ofrece el núcleo.
- **2. La interfaz pertenece a quien la necesita:** En Onion la interfaz la escribe el núcleo según lo que necesita, con nombres del dominio (ej. GetBySku y no ExecuteQuery). Cockburn (2005) expresa que el protocolo de un puerto lo da el propósito de la conversación.
- **3. La regla la garantiza el compilador:** Cada anillo puede ser un proyecto distinto y las referencias solo apuntan hacia adentro (Domain no referencia a nadie; Application referencia a Domain; Infrastructure referencia a Application/Domain).
- **4. La conexión se hace en la raíz de composición:** El único lugar que conoce a la vez los puertos y sus adaptadores es el punto de arranque de la aplicación, situado en el anillo exterior.

```
// Inventory.Domain (centro: puerto de salida del dominio)
public interface IProductRepository
{
    Product? GetBySku(string sku);
    void Save(Product product);
}

// Inventory.Application (puerto de salida técnico y puerto de entrada)
public interface INotificationSender
{
    void NotifyLowStock(string sku, int stock);
}

public class InventoryService : IInventoryService
{
    private readonly IProductRepository _products;
    private readonly INotificationSender _notifier;

    public InventoryService(IProductRepository products, INotificationSender
notifier)
        => (_products, _notifier) = (products, notifier);

    public void RegisterExit(string sku, int quantity)
```

```
    {
        var product = _products.GetBySku(sku) ?? throw new
ProductNotFoundException(sku);
        product.RegisterExit(quantity);
        _products.Save(product);

        if (product.IsBelowMinimum)
            _notifier.NotifyLowStock(product.Sku, product.Stock);
    }
}

// Inventory.Infrastructure (anillo exterior: adaptadores de salida)
public class SqlProductRepository : IProductRepository { /* EF Core */ }
public class SmtpNotificationSender : INotificationSender { /* SMTP */ }

// Inventory.Web (anillo exterior: adaptador de entrada)
[ApiController, Route("api/inventory")]
public class InventoryController : ControllerBase
{
    private readonly IInventoryService _service;
    public InventoryController(IInventoryService service) => _service = service;

    [HttpPost("{sku}/exit")]
    public IActionResult Exit(string sku, int quantity)
    {
        _service.RegisterExit(sku, quantity);
        return NoContent();
    }
}

// Raíz de composición (arranque de la aplicación)
services.AddScoped<IInventoryService, InventoryService>();
services.AddScoped<IProductRepository, SqlProductRepository>();
services.AddScoped<INotificationSender, SmtpNotificationSender>();
```

c) ¿Por qué el hexágono no tiene seis lados por una razón numérica?

Cockburn (2005) lo dice expresamente: el hexágono no es un hexágono porque el número seis sea importante, sino para que quien dibuja tenga espacio para insertar los puertos y adaptadores que necesite, sin estar limitado por un dibujo en capas unidimensional. Añade que el nombre «arquitectura hexagonal» proviene únicamente de ese efecto visual. El nombre que describe el contenido del estilo es Puertos y Adaptadores.

De ahí se deducen varias ideas:

- **El número de caras es arbitrario:** Una aplicación puede tener dos puertos o diez. Una cara no representa una tecnología concreta, sino un espacio donde situar un puerto.
- **Una figura sin arriba ni abajo:** Un polígono regular se puede girar sin que cambie su significado, porque en el hexágono la posición no indica nada sobre las dependencias. Lo único que importa es si algo está dentro o fuera.

- **Se cambia la asimetría importante:** La asimetría que sí importa es la de interior y exterior: el interior contiene las reglas de la aplicación y el exterior todo lo demás, y la dependencia siempre va de fuera hacia dentro.
- **Matiz de los dos lados:** Cockburn (2005) suele colocar a la izquierda los actores que dirigen la aplicación (primarios) y a la derecha los dirigidos por ella (secundarios). La distinción describe quién inicia la conversación, no quién depende de quién.

Pregunta 3: División del Sistema en Contextos Delimitados (Bounded Contexts)

Para abordar la complejidad de una empresa de seguros donde distintos departamentos (actuarios, facturación, ajustadores de siniestros) utilizan un mismo término como "póliza" con significados y preocupaciones completamente diferentes, se deben aplicar los conceptos de diseño guiado por el dominio y contextos delimitados.

Un contexto delimitado (bounded context) define explícitamente las fronteras de un modelo, de modo que todo lo relacionado con el dominio es visible internamente pero permanece opaco para otros contextos (Richards & Ford, 2020).

Basado en lo anterior, el sistema de gestión de pólizas de automóviles se dividiría en los siguientes contextos delimitados:

- **Contexto Delimitado de Pólizas:** Se enfoca en la perspectiva actuarial, gestionando el contrato, las cláusulas y las condiciones del seguro.
- **Contexto Delimitado de Facturación de Primas:** Trata la póliza exclusivamente como una fuente de ingresos, gestionando cobros, primas y transacciones financieras.
- **Contexto Delimitado de Siniestros y Evaluación de Daños:** Opera con la aplicación móvil de los ajustadores, donde la póliza funge como un número de referencia para registrar y evaluar los daños ocurridos en el lugar del accidente.

Prevención de la Contaminación del Modelo de "Póliza"

Para evitar que el modelo central de "Póliza" se contamine con los detalles operativos de facturación o siniestros, se debe rechazar la creación de un modelo unificado o entidades compartidas a nivel global en toda la organización.

En su lugar, cada contexto delimitado mantiene su propio modelo especializado y coherente, asegurando que las preocupaciones de un área (como los ingresos por facturación o los números de referencia de siniestros) no filtren lógica ni alteren la estructura del contrato principal (Richards & Ford, 2020).

El aislamiento de los componentes y la alta cohesión dentro de cada contexto protegen la integridad conceptual del dominio, evitando acoplamientos indeseados entre diferentes áreas de negocio.

Comunicación Desacoplada entre Contextos

Para lograr que los contextos delimitados se comuniquen de manera desacoplada, se deben evitar las llamadas síncronas rígidas que incrementan la connascencia dinámica y el acoplamiento temporal entre los subsistemas (Richards & Ford, 2020).

El uso de mensajería asincrónica (como colas o tópicos de publicación y suscripción) permite que los contextos intercambien información de forma desacoplada, amortiguando diferencias operativas y de carga entre las partes (Richards & Ford, 2020).

En los puntos de integración donde un contexto requiera datos de otro, se deben emplear contratos claros y mecanismos de traducción que eviten la propagación de dependencias estructurales directas entre los diferentes servicios y módulos (Richards & Ford, 2020).

Relación con la Arquitectura Orientada al Dominio (Clean / Onion)

En el diseño guiado por el dominio, hay tres usos básicos que determinan la elección de un modelo:

- **1.** El modelo dicta la forma del diseño del núcleo del software. Es el vínculo íntimo entre el modelo y la implementación lo que hace que el modelo sea relevante y garantiza que el análisis invertido en él se aplique al producto final, un programa en ejecución. Esta vinculación del modelo y la implementación también ayuda al mantenimiento y al desarrollo continuo porque el código se puede interpretar a partir de la comprensión del modelo (Evans, 2003).
- **2.** El modelo es la columna vertebral de un lenguaje utilizado por todos los miembros del equipo. Debido a la vinculación del modelo y la implementación, los desarrolladores pueden hablar sobre el programa en este lenguaje. Pueden comunicarse con expertos en el dominio sin traducción. Y debido a que el lenguaje se basa en el modelo, nuestras habilidades lingüísticas naturales pueden orientarse a refinar el propio modelo (Evans, 2003).
- **3.** El modelo es conocimiento destilado. El modelo es la forma acordada por el equipo para estructurar el conocimiento del dominio y distinguir los elementos de mayor interés. Un modelo captura cómo elegimos pensar sobre el dominio a medida que seleccionamos términos, descomponemos conceptos y los relacionamos. El lenguaje compartido permite una colaboración eficaz entre desarrolladores y expertos en el dominio para estructurar la información en esta forma. La vinculación del modelo y la implementación garantiza que la experiencia con las primeras versiones del software sea también una retroalimentación válida para el proceso de modelado (Evans, 2003).
- **El Modelo de Dominio como Núcleo Central:** En la Guía de Arquitectura N-Capas orientada al dominio, se establece que el modelo de dominio es el corazón del software y debe aislarse completamente del resto de capas, aplicando el principio de ignorancia de persistencia (Persistence Ignorance). Esto guarda una relación directa con Clean Architecture y la arquitectura Onion, donde las reglas de negocio críticas (entidades y casos de uso) ocupan el centro absoluto del diseño.
- **Inversión de Dependencias y Contratos:** La guía propone que los contratos o interfaces de los repositorios se definan estrictamente dentro de la Capa de Dominio, mientras que su implementación técnica mediante un ORM (como Entity Framework) resida en la Capa de

Infraestructura. Esto implementa directamente el Principio de Inversión de Dependencias (DIP), garantizando que los detalles técnicos dependan de las abstracciones del dominio y no a la inversa.

- **Los Detalles como Elementos Periféricos:** Tal como se postula en Clean Architecture al indicar que "la base de datos es un detalle" y "los frameworks son herramientas", la arquitectura N-Capas separa los mecanismos de infraestructura para que las decisiones tecnológicas puedan ser diferidas o reemplazadas sin comprometer la lógica de negocio.

Creación de un Adaptador de Servicio para la Comunicación Externa

- **Agentes de Servicios Externos:** La Guía de Arquitectura N-Capas define los Agentes de Servicios Externos dentro de la capa de infraestructura de datos para gestionar la semántica de comunicación con servicios web u otros sistemas back-end.
- **Patrón Adaptador:** Estos agentes operan como adaptadores que encapsulan y aíslan la idiosincrasia de los servicios externos, encargándose además de realizar las transformaciones y mapeos necesarios entre los distintos formatos de datos.
- **Bajo Acoplamiento y Sustitución:** Al apoyarse en interfaces estables, el uso de estos agentes como adaptadores externos permite que un servicio de terceros pueda ser modificado o sustituido por una implementación completamente diferente sin que el sistema interno o el dominio se vean afectados.

Pregunta 4: Evaluación de la Inversión de Dependencias en la Práctica

¿Por qué la dependencia directa viola los principios de las arquitecturas Hexagonal, Limpia y Onion?

En el diseño original, OrderProcessor instancia y llama directamente a AddressValidatorService, que a su vez depende de una librería HTTP específica. El flujo de control va de la lógica de negocio hacia el servicio externo, y las dependencias del código fuente van en la misma dirección. Es decir, un componente de alto nivel (las reglas para procesar pedidos) depende de un componente de bajo nivel (la comunicación técnica con un proveedor).

El Principio de Inversión de Dependencias establece que los módulos de alto nivel no deben depender de los de bajo nivel, y que ambos deben depender de abstracciones. Además, las abstracciones no deben depender de los detalles, sino al contrario (Martin, 2017). Con la dependencia directa, OrderProcessor conoce la clase concreta, y a través de ella la librería HTTP. Cambiar de librería o de proveedor podría obligar a modificar la lógica de negocio, aunque las reglas de los pedidos sigan igual.

La Regla de Dependencia de la Arquitectura Limpia dice que las dependencias del código fuente solo pueden apuntar hacia adentro, hacia las políticas de más alto nivel, y que nada en un círculo interior debe conocer nada de un círculo exterior (Martin, 2017). Los servicios web, las bases de datos y los frameworks son detalles que viven en el exterior. Aquí ocurre lo contrario: el núcleo conoce el detalle.

Con las otras dos arquitecturas pasa lo mismo:

- **Hexagonal:** el núcleo de la aplicación debe comunicarse con el exterior a través de puertos, y cada tecnología concreta se conecta mediante un adaptador (Cockburn, 2005). Si OrderProcessor usa directamente la clase concreta, no existe puerto y el núcleo queda atado a una tecnología.
- **Onion:** el modelo de dominio está en el centro, todo el acoplamiento va hacia el centro y la infraestructura queda en la capa externa, implementando interfaces definidas en las capas internas (Palermo, 2008). Aquí una capa interna depende de una externa.

La dirección correcta es esta: la infraestructura (AddressValidatorService) debe depender de una abstracción definida por el núcleo, y el núcleo no debe depender de la infraestructura.

Rediseño de la interacción

La solución es invertir la relación con una interfaz que exprese lo que la aplicación necesita, sin decir cómo se obtiene.

- **Dentro del núcleo (capa de aplicación):** IAddressValidator y OrderProcessor. La interfaz es el puerto de salida (Cockburn, 2005) y la define el núcleo según sus necesidades.
- **Fuera del núcleo (infraestructura):** AddressValidatorService, que implementa la interfaz y funciona como adaptador. Es el único componente que conoce HTTP y el proveedor.

Esta idea coincide con lo que plantea la Guía de Arquitectura N-Capas orientada al Dominio: la lógica de dominio no debe conocer los detalles de acceso a recursos externos, y para ello se usan abstracciones con inyección de dependencias y adaptadores en la capa de infraestructura (De la Torre et al., 2010).

Aunque el flujo de ejecución sigue yendo de OrderProcessor al servicio externo, la dependencia del código fuente queda invertida: la flecha de AddressValidatorService apunta hacia la interfaz del núcleo.

```
// Núcleo (capa de aplicación)
public interface IAddressValidator
{
    Task<bool> IsValidAsync(string address);
}

public class OrderProcessor
{
    private readonly IAddressValidator _validator;

    public OrderProcessor(IAddressValidator validator)
    {
        _validator = validator;
    }

    public async Task<bool> ProcessOrderAsync(string address)
    {
        if (!await _validator.IsValidAsync(address))
            return false;

        // Resto de las reglas de procesamiento del pedido.
        return true;
    }
}
```

```
// Infraestructura (adaptador)
public class AddressValidatorService : IAddressValidator
{
    private readonly HttpClient _httpClient;

    public AddressValidatorService(HttpClient httpClient)
    {
        _httpClient = httpClient;
    }

    public async Task<bool> IsValidAsync(string address)
    {
        var response = await _httpClient.GetAsync("addresses/validate?address=" +
Uri.EscapeDataString(address));
        response.EnsureSuccessStatusCode();
        return await response.Content.ReadFromJsonAsync<bool>();
    }
}
```

La ruta, el formato de respuesta y el manejo de errores son ilustrativos y deben ajustarse a la API real. La implementación concreta se registraría en el contenedor de inyección de dependencias en la raíz de composición de la aplicación.

Justificación: El diseño cumple que los detalles dependan de las abstracciones (Martin, 2017). Además separa responsabilidades: OrderProcessor se ocupa de los pedidos y el adaptador de la comunicación externa.

Pruebas aisladas, mantenibilidad y cambio de proveedor

Pruebas aisladas: OrderProcessor recibe un IAddressValidator, por lo que en las pruebas unitarias se puede inyectar un doble de prueba en lugar del adaptador real:

```
public class FakeAddressValidator : IAddressValidator
{
    private readonly bool _result;
    public FakeAddressValidator(bool result) => _result = result;
    public Task<bool> IsValidAsync(string address) => Task.FromResult(_result);
}

// Dirección válida: el pedido continúa.
var ok = await new OrderProcessor(new FakeAddressValidator(true))
    .ProcessOrderAsync("Calle Principal 123");  // se espera true

// Dirección inválida: el pedido se detiene.
var fail = await new OrderProcessor(new FakeAddressValidator(false))
    .ProcessOrderAsync("Dirección incorrecta"); // se espera false
```

En una prueba real, esos resultados esperados se comprobarían con aserciones del framework de pruebas. Estas pruebas no hacen llamadas HTTP, no dependen de la red ni de la disponibilidad del proveedor, y son rápidas y reproducibles. Esa es la ventaja de que el núcleo de la aplicación sea comprobable sin la UI, la base de datos ni ningún elemento externo (Martin, 2017; Cockburn, 2005). Las pruebas con dobles no sustituyen a las de integración, que siguen siendo necesarias para verificar que el adaptador funciona con el servicio real.

Mantenibilidad y cambio de proveedor: Cambiar de proveedor se reduce a escribir otro adaptador que implemente IAddressValidator y registrarlo en la configuración de dependencias. Lo mismo ocurre si solo se sustituye la librería HTTP, porque el cambio queda concentrado en el adaptador. Así los detalles se comportan como «plugins» del núcleo (Martin, 2017) y se reduce el riesgo de introducir errores en la lógica de pedidos.

Esta independencia tiene límites. Si el nuevo proveedor cambia el significado de la validación o los datos que la aplicación necesita, habría que revisar la interfaz y quizás parte de la lógica. La interfaz protege frente a cambios técnicos, pero no frente a cambios en los requisitos funcionales.

Pregunta 5: Análisis de un problema real: el Kitty Problem y el Principio de Abierto/Cerrado

Descripción del Kitty Problem

El Kitty Problem es un ejemplo que presenta Martin (2017, cap. 27) para ilustrar dos falacias asociadas a los servicios: la falacia del desacoplamiento y la falacia del desarrollo y despliegue independiente. Se parte de un sistema agregador de taxis que conoce a varios proveedores de una ciudad y permite a los clientes pedir viajes según criterios como el tiempo de recogida, el costo, el lujo y la experiencia del conductor. El sistema está construido con microservicios:

- **TaxiUI:** atiende a los clientes, que piden los taxis desde sus dispositivos móviles.
- **TaxiFinder:** examina los inventarios de los distintos TaxiSuppliers y determina qué taxis son candidatos para el usuario.
- **TaxiSelector:** toma los criterios del usuario y elige un taxi apropiado entre los candidatos.
- **TaxiDispatcher:** recibe el taxi elegido y lo ordena.

Después de más de un año de operación, el departamento de marketing anuncia un servicio de entrega de gatitos a domicilio. Se habilitarán varios puntos de recogida en la ciudad y, cuando se haga un pedido, se seleccionará un taxi cercano que recoja el gatito y lo entregue en la dirección indicada. Solo un proveedor participa al inicio y otros podrían sumarse más adelante. Además, existen restricciones por alergias: no deben seleccionarse conductores alérgicos a los gatos, ni vehículos que hayan transportado gatitos en los últimos tres días para clientes que declaren alergia.

Martin (2017) señala que, para implementar esta funcionalidad, tendrían que cambiar todos los servicios, por lo que su desarrollo y despliegue deberían coordinarse con mucho cuidado. En otras palabras, los servicios están acoplados y no pueden desarrollarse, desplegarse ni mantenerse de forma independiente.

¿Por qué la división en servicios basados en verbos o entidades funcionales llevó a este problema?

- **La falacia del desacoplamiento:** los servicios están desacoplados a nivel de variables, pero siguen acoplados por los recursos compartidos y, sobre todo, por los datos que intercambian. Martin (2017) da como ejemplo que, si se añade un campo a un registro que circula entre servicios, todos

los que operan sobre ese campo deben cambiar y deben coincidir en cómo interpretarlo. Además, las interfaces de los servicios no son más formales ni mejor definidas que las de las funciones.

- **La falacia del desarrollo y despliegue independiente:** se supone que cada servicio puede ser desarrollado y operado por un equipo propio. Sin embargo, en la medida en que los servicios están acoplados por datos o por comportamiento, su desarrollo y despliegue deben coordinarse (Martin, 2017).
- **La descomposición funcional:** Martin (2017) explica que las descomposiciones funcionales, como la de TaxiUI, TaxiFinder, TaxiSelector y TaxiDispatcher, son muy vulnerables a las nuevas funcionalidades que atraviesan todos esos comportamientos, y que este problema afecta a cualquier sistema, sea orientado a servicios o no. Esto se parece a la partición técnica que describen Richards y Ford (2020), en la que los componentes se organizan por su papel dentro del flujo y no por el dominio del negocio.
- **Los servicios no definen por sí solos la arquitectura:** según Martin (2017), los servicios son, en esencia, llamadas a funciones a través de fronteras de procesos o plataformas. La arquitectura queda definida por las fronteras que separan las políticas de alto nivel de los detalles y que siguen la Regla de Dependencia, y no por la forma física en que los elementos se comunican o se ejecutan.

¿Cómo se podría haber diseñado el sistema para que el cambio afectara a un solo componente?

Martin (2017) plantea que, si se hubieran considerado los principios SOLID, se habría creado un conjunto de clases que pudieran extenderse de forma polimórfica para manejar nuevas funcionalidades. Las clases del modelo corresponden a grandes rasgos con los servicios originales, pero con fronteras bien definidas y dependencias que siguen la Regla de Dependencia:

- **Clases base abstractas:** conservan gran parte de la lógica de los servicios originales.
- **Componente Rides:** contiene la lógica específica de los viajes, extraída de los servicios originales.
- **Componente Kittens:** contiene la nueva funcionalidad de entrega de gatitos.

Los componentes Rides y Kittens sobrescriben las clases base abstractas mediante patrones como Template Method o Strategy, y las clases que implementan cada funcionalidad son creadas por fábricas bajo el control de la interfaz de usuario. De esta forma, al implementar la funcionalidad de los gatitos, solo TaxiUI debe cambiar. Lo demás se resuelve añadiendo un nuevo componente (un archivo jar, una gema o una DLL) que se carga dinámicamente en tiempo de ejecución. Así, la funcionalidad queda desacoplada y puede desarrollarse y desplegarse de forma independiente (Martin, 2017).

Martin (2017) añade que esta solución también se aplica a los servicios: estos no tienen que ser pequeños monolitos, sino que pueden diseñarse con los principios SOLID y con una estructura interna de componentes, de modo que se agreguen componentes nuevos sin modificar los existentes. La conclusión es que las fronteras arquitectónicas no pasan entre los servicios, sino a través de ellos, dividiéndolos en componentes.

Esta propuesta coincide con la lógica de las arquitecturas orientadas al dominio. Tanto en Onion como en Clean las dependencias apuntan hacia el núcleo, y los detalles se implementan en capas externas a partir de abstracciones definidas hacia adentro (Palermo, 2008; Martin, 2017). Además, organizar el

sistema alrededor de conceptos del dominio, como las entregas de gatitos y los viajes, y no alrededor de pasos técnicos, es coherente con el enfoque de DDD (De la Torre Llorente et al., 2010).

Relación con el Principio de Abierto/Cerrado (OCP)

El OCP establece que una entidad de software debe estar abierta a la extensión y cerrada a la modificación. En el diseño propuesto, añadir una funcionalidad se reduce a incorporar nuevo código que extiende las clases abstractas existentes, sin modificar el código que ya funciona. Martin (2017) lo resume al indicar que agregar nuevas funcionalidades se ajusta al Principio de Abierto/Cerrado: las clases base y los demás componentes permanecen cerrados a la modificación, mientras que el sistema queda abierto a la extensión mediante nuevos componentes como Kittens.

El papel de los Adaptadores

Martin (2017) no utiliza el término adaptador en este capítulo, pero la solución puede leerse a la luz de Puertos y Adaptadores (Cockburn, 2005). Las clases base abstractas cumplen el papel de puertos, es decir, los contratos que define el interior del sistema. Los componentes Rides y Kittens funcionan como adaptadores, porque implementan esos contratos y traducen las reglas propias de cada funcionalidad a lo que el núcleo espera. Las fábricas controladas por la UI deciden qué adaptador se conecta en cada caso. Gracias a esto, la extensión ocurre del lado de los adaptadores y el núcleo no se modifica.

Conclusión

El Kitty Problem muestra que el origen del problema no es la cantidad de servicios, sino el lugar donde se trazan las fronteras. Si el sistema se divide únicamente por pasos funcionales, un requisito transversal obliga a cambiar todos los servicios. Si, en cambio, cada servicio se diseña con una estructura interna de componentes que respeta la Regla de Dependencia y las variaciones entran como componentes que extienden abstracciones, el cambio se concentra en un único componente nuevo y el resto del sistema permanece intacto.

Pregunta 6: Evaluación Crítica de Capas Abiertas y Fast-Lane Readers

La Arquitectura N-Capas es criticada por su bajo rendimiento en características como “Agilidad general”, “Facilidad de despliegue” y “Escalabilidad” (Richards, 2015). Un estudiante propone que, con el uso de capas abiertas (open layers) y técnicas como el “fast-lane reader”, se pueden mitigar estos problemas.

El uso de capas abiertas y técnicas como el fast-lane reader funciona únicamente como un “mecanismo de escape” de optimización de rendimiento a corto plazo. No es una solución sostenible para mitigar las deficiencias de la Arquitectura N-Capas ya que el pequeño beneficio en agilidad general se paga con una degradación crítica en la mantenibilidad, el despliegue y la escalabilidad del sistema.

Para demostrar esto, podemos analizar el impacto de su propuesta en las tres dimensiones evaluadas:

- **Agilidad general:** El uso de capas abiertas y el patrón fast-lane reader mejoran la agilidad en flujos simples. Como explican Richards y Ford (2020), esta técnica se utiliza porque resulta mucho más rápido y fácil que la capa de presentación acceda a la base de datos de forma directa para solicitudes sencillas de recuperación, omitiendo los niveles innecesarios intermediarios.

- **Escalabilidad:** La propuesta perjudica severamente la escalabilidad a largo plazo. Al abrir una capa, la capa superior pasa a depender directamente de los contratos de las capas inferiores. Richards (2015) advierte sobre las consecuencias de este acoplamiento directo señalando que cualquier modificación técnica o cambio estructural realizado en un nivel inferior se propagará inevitablemente en cascada a través de las otras capas de la arquitectura. Si se necesita realizar un cambio en una capa de abajo para escalar el sistema, el impacto obliga a modificar y reescribir código en todas las capas superiores.
- **Facilidad de despliegue:** Desplegar el sistema se vuelve mucho más difícil porque los componentes ya no son independientes. Al abrir capas y conectar la presentación con la persistencia mediante un fast-lane reader se crea un bloque rígido de dependencias entrelazadas. Richards y Ford (2020) detallan este problema al explicar que la apertura rompe el principio de las capas de aislamiento. Como los componentes superiores ahora dependen de los inferiores de forma directa, todas las capas (desde el nivel de la capa abierta hacia arriba) se vuelven dependientes unas de las otras y deben ser compiladas, probadas y desplegadas juntas obligatoriamente.

Por eso podemos afirmar que la solución del estudiante no es correcta más allá de la agilidad general.

¿Cómo afecta la introducción de capas abiertas al principio de Aislamiento de Capas y a la mantenibilidad a largo plazo?

El concepto de aislamiento de capas implica que los cambios realizados en una capa de la arquitectura, por regla general, no repercuten ni afectan a los componentes de otras capas, siempre y cuando los contratos entre dichas capas permanezcan inalterados. Cada capa es independiente de las demás, por lo que posee un conocimiento escaso o nulo sobre el funcionamiento interno de las otras capas de la arquitectura. No obstante, para garantizar este aislamiento arquitectónico, Richards y Ford (2020) señalan que los componentes que forman parte del flujo principal de las peticiones del sistema deben permanecer necesariamente cerrados.

Supongamos que tenemos una variante de arquitectura con cuatro capas: presentación, negocio, persistencia y base de datos. Si decidimos que la capa de negocio sea abierta, permitimos que la presentación se salte el negocio para comunicarse directamente con la persistencia.

De esta forma, la capa de presentación va a depender tanto de la capa de negocio como de la capa de persistencia, lo que significa que existen contratos bien definidos <presentación, negocio> y <presentación, persistencia>. Si en un momento dado tuviéramos que realizar un cambio de estructura en la capa de persistencia, dicho cambio no sería absorbido por la capa de negocio directamente, sino que repercutiría también en la capa de presentación (efecto dominó). Esto rompe por completo el principio de aislamiento y hace más vulnerable la mantenibilidad a largo plazo porque un cambio en un nivel inferior nos obliga a revisar y reescribir código en más de un nivel superior del sistema.

¿Por qué la solución de capas abiertas puede llevar a un “sinkhole anti-pattern” y qué dice la regla del 80/20 sobre esto?

El problema del sinkhole en realidad pasa cuando usamos capas cerradas. Las capas abiertas son la solución que usamos para arreglarlo. Si usamos capas cerradas, una solicitud puede verse obligada a bajar desde la capa de arriba hasta la base de datos sin que nadie le haga nada en el camino. Los datos

viajan sin más por el código y luego regresan vacíos. Esto es el antipatrón architecture sinkhole. Según Richards (2015), esto ocurre cuando las solicitudes transitan a través de los diferentes niveles arquitectónicos funcionando como simples canales de paso, sin que se ejecute ningún tipo de lógica de negocio o procesamiento real dentro de ellos.

Richards (2015) dice que es normal que esto pase un poco en los sistemas. Ahí es donde entra su regla del 80/20:

Si la gran mayoría de tus solicitudes (el 80%) sí hacen lógica de negocio y solo un 20% son simples consultas vacías, no pasa nada. El sistema está sano y es mejor dejar las capas cerradas para proteger el código.

Por otra parte, Richards (2015) explica que si se llega a detectar que el 80% o una proporción mayor de las peticiones totales del sistema se convierten en sumideros, esto representa una advertencia evidente de que se deben abrir ciertas capas de forma selectiva o evaluar un cambio completo hacia otro estilo de arquitectura.

Entonces, si tienes demasiados sumideros (80%), abres la capa para que los datos vayan directo y el sistema sea más rápido. Lo malo es que al hacer esto, alivias el sinkhole pero rompes el aislamiento de las capas y el mantenimiento se vuelve más difícil en el futuro.

¿Por qué las Arquitecturas Limpia y Hexagonal son inherentemente más adecuadas?

Las arquitecturas Hexagonal y Limpia son inherentemente más adecuadas para sistemas con requisitos de alta escalabilidad y agilidad porque eliminan el acoplamiento vertical desde su diseño, algo que las N-Capas no pueden lograr ni siquiera con optimizaciones.

La diferencia se vuelve clara al analizar cómo maneja cada arquitectura sus principios fundamentales:

- **El problema del acoplamiento en N-Capas:** En una arquitectura de N-Capas tradicional, la capa de negocio depende directamente de la capa de la base de datos. Si tuviéramos que cambiar la base de datos para escalar el sistema, la capa de negocio se vería afectada y tendríamos que hacer cambios en ella también. Esto frena por completo la agilidad, porque tocar una capa de abajo te obliga a reescribir código en la capa de encima, generando un efecto dominó.
- **La inmunidad del negocio (Clean Architecture):** Al aplicar la Arquitectura Limpia de Martin (2017), el negocio se coloca en el centro y deja de depender de los frameworks o las bases de datos. Si decidimos cambiar la base de datos o el framework, solo modificamos esas herramientas externas sin tener que tocar una sola línea del código de negocio. Como explica Martin (2017) en su análisis sobre los límites arquitectónicos, el diseño consiste en establecer fronteras claras que separen los elementos de software entre sí, impidiendo de forma estricta que los componentes ubicados en un lado tengan conocimiento o dependan de los que se encuentran en el otro. Al separar el negocio de la tecnología exterior, el sistema se adapta a los cambios con mucha más agilidad.
- **Escalabilidad técnica sin romper nada (Hexagonal):** Siguiendo el modelo de Cockburn (2005), el "adentro" (el negocio) se comunica con el "afuera" mediante puertos y adaptadores. Si el sistema necesita crecer para soportar millones de usuarios, basta con crear un nuevo adaptador para una base de datos más potente que respete el contrato existente. Así, la nueva base de datos se conecta

sin alterar el código del negocio. De acuerdo con Cockburn (2005), este enfoque garantiza que la aplicación central sea desarrollada y ejecutada de manera totalmente aislada respecto a sus dispositivos, bases de datos o infraestructura de tiempo de ejecución finales.

- **Agilidad en el equipo de desarrollo:** Estas arquitecturas permiten un verdadero aislamiento gracias a sus interfaces. Esto ayuda enormemente al equipo de desarrollo, ya que permite hacer cambios en una capa sin tener que tocar la otra, siempre y cuando se respete el contrato. Los programadores pueden trabajar en paralelo usando datos simulados (mocks), aumentando drásticamente la velocidad de entrega.

Entonces, optimizar una arquitectura N-Capas (como abrir capas) es solo poner un parche. A la larga, parchar las N-Capas hace que su mantenimiento sea mucho más difícil y consume más recursos. Por el contrario, las arquitecturas Limpia y Hexagonal no necesitan parches, nacen preparadas para la agilidad y la alta escalabilidad porque protegen el dominio del negocio desde su propia estructura.

Referencias Bibliográficas

Cockburn, A. (2005). Hexagonal Architecture (Ports and Adapters). https://alistair.cockburn.us/hexagonal-architecture/

De la Torre Llorente, C., Zorrilla Castro, U., Ramos Barroso, M. A., & Calvarro Nelson, J. (2010). Guía de Arquitectura N-Capas orientada al Dominio con .NET 4.0 (Borrador). Microsoft Ibérica.

Evans, E. (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley Professional.

Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.

Palermo, J. (2008). The Onion Architecture. https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/

Richards, M. (2015). Software Architecture Patterns. O'Reilly Media.

Richards, M., & Ford, N. (2020). Fundamentals of Software Architecture: An Engineering Approach. O'Reilly Media.

Declaración Individual sobre el Uso de Inteligencia Artificial

-Para la elaboración de las respuestas a las preguntas 1 y 2 de este seminario se utilizó la herramienta Claude (modelo Claude Opus 3.5, de Anthropic). El integrante Raciel Pupo Santos, responsable de ambas preguntas utilizó Claude para mejorar la estructura y redacción de la argumentación a partir de la bibliografía recomendada, ayudar a generar los ejemplos de código en C# y los diagramas en TikZ.

-En la realización de la pregunta 3 y el documento final, el estudiante Ramón Hernández Camacho hizo uso de Gemini para labores de traducción, unificación de los documentos, correcciones de formato, revisión ortográfica; además, la empleó para entender mejor conceptos mientras usaba la bibliografía y en labores de resumen.

-Para la elaboración de la respuesta a la pregunta 4 de este seminario se utilizó la herramienta Claude (modelo Claude Sonnet 3.5, de Anthropic). El integrante que la empleó fue Leandro Márquez Blanco, responsable de la pregunta. El estudiante elaboró primero una solución propia, que se proporcionó a la herramienta junto con el enunciado del seminario, y Claude se utilizó para mejorar la redacción y la estructura de la respuesta, ampliar la argumentación con la bibliografía recomendada y generar los ejemplos de código en C#.

-Para la respuesta a la pregunta 5 se utilizó la herramienta Claude (modelo Claude Sonnet 3.5, de Anthropic). Los propósitos fueron los siguientes: Verificación de la fuente**:** identificar que el Kitty Problem proviene del capítulo 27 de Clean Architecture (Martin, 2017) y no de Fundamentals of Software Architecture, como indica el enunciado. También traducir al español y parafrasear los pasajes del capítulo 27 de Clean Architecture, que está en inglés y cuyo texto se extrajo del libro. Además organizar la respuesta y adaptarla al estilo y formato de las respuestas del equipo. Por último se empleó en la generación de código LaTeX para elaborar el código fuente de este documento.

El borrador generado se contrastó con el texto original del libro y se revisó y ajustó antes de su inclusión. La relación entre la solución de Martin y el modelo de Puertos y Adaptadores (Cockburn, 2005), así como el vínculo con la partición técnica de Richards y Ford (2020), fue una sugerencia de la herramienta que verificó con las fuentes citadas.

- Se usó el modelo de ChatGPT-6 Luna. El propósito de su uso fue entender las normas APA y corregir el citado y las referencias bibliográficas. Al modelo se le pidió que ayudara con ejemplos de diferentes tipos de citados y referencias bibliográficas, entendiendo de esta manera las diferentes formas de citar.